# Blockchain Empowered Differentially Private and Auditable Data Publishing in Industrial IoT

Lei Xu, Ting Bao, and Liehuang Zhu, *Member, IEEE*

***Abstract*—As more and more organizations deploy their sensing devices in the industrial Internet of Things network, it becomes increasingly important for the organizations to share data with others, so that the value of the data can be fully explored. However, individuals' privacy may be compromised because of data sharing. In this article, we study the differentially private data publishing problem, which aims at balancing between privacy and data utility. Specifically, two blockchain-based data publishing protocols are proposed. For histogram publishing, we propose a protocol where the Laplace noise added in the query result is verified by the blockchain. For anonymized data publishing, we propose a protocol, which can prevent the publisher and the recipient from lying about the utility of the published data. With the blockchain acting as a reliable intermediary between the publisher and the recipient, the proposed protocols can help to realize fair and auditable data sharing.**

***Index Terms*—Blockchain, differential privacy, histogram publishing, transaction verification, verifiable shuffle.**

## I. Introduction

INDUSTRIAL Internet of Things (IIoT) is a system consisted of highly connected smart industrial components, cyber-physical assets, and optional cloud or edge computing platforms [1], [2]. IIoT enables real-time monitoring and autonomous controlling of industrial processes, thus, can help to improve the overall performance of industrial systems. The large amount of data generated by the sensors or actuators deployed in industrial environment are the cornerstone of IIoT. Since an industrial process generally involves the collaboration of multiple parties, currently there is an urgent need for secure and reliable data sharing schemes.

Data from the sensing devices may contain sensitive information of individuals. Sharing data among different parties may cause privacy issues. For example, a smart meter deployed at an individual's house collects data about household electricity consumption. Such data will reveal the individual's private information such as household occupancy and economic status [3]. Suppose that a research institute wants to analyze the pattern of electricity consumption in a certain area, and it expects the power company to provide a sufficient amount of readings collected from smart meters. If the power company directly shares the readings with the institute, its customers' privacy will be compromised. A data sharing scheme is inoperable unless it handles the privacy issue properly.

The emerging technology blockchain has attracted much attention from the Internet of Things (IoT) field in recent years [4]. Essentially, blockchain is a decentralized database, which maintains a continuously growing list of transaction records in a verifiable way [5]. Researchers have explored how to utilize blockchain to deal with the security and privacy issues in IoT applications [6]. When applied to data sharing scenarios, blockchain is mainly used to record the access policies and the interactions between different parities. The tamper-proof records can help to audit the data accessing behaviors or to verify the integrity of the shared data [7]. The blockchain itself cannot solve the privacy issue caused by data sharing. It is usually used together with some other privacy-preserving technique. For example, in the data sharing scheme proposed in [8], the sensing data from a mobile terminal are first encrypted and then sent to the public blockchain network. In the blockchain-based data sharing architecture proposed by Lu *et al.* [9], the sensing data are not directly shared among different parties. Instead, the parities adopt a federated learning approach, and only the trained models are shared via the blockchain network. In this way, the privacy of each party's data can be preserved.

Different from the studies mentioned previously, in this article, we consider a scenario where the data collected from sensing devices cannot be shared in the form of incomprehensible ciphertexts or model parameters. This scenario is common in practice. Using the aforementioned smart meter example again. If the research institute wants to keep its analysis method as a secret but is not able to analyze ciphertexts, then the readings provided by the power company should have similar forms with the original readings. As shown in Fig. 1, in a typical edge-cloud architecture of IIoT, the edge servers, which collect data from sensing devices are responsible to protect individuals' privacy. To share useful data with others without disclosing much information about any specific individual, the edge server can apply a differentially private mechanism.

Manuscript received July 29, 2020; revised December 12, 2020; accepted December 12, 2020. Date of publication December 15, 2020; date of current version July 26, 2021. This work was supported in part by the Natural Science Foundation of China under Grant 61871037, in part by the Natural Science Foundation of Beijing, China under Grant M21035, and in part by the Beijing Institute of Technology Research Fund Program for Young Scholars. Paper no. TII-20-3654. *(Corresponding author: Liehuang Zhu.)*

The authors are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: 6120180029@bit.edu.cn; btweifar@126.com; liehuangz@bit.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2020.3045038.

Digital Object Identifier 10.1109/TII.2020.3045038

1551-3203 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

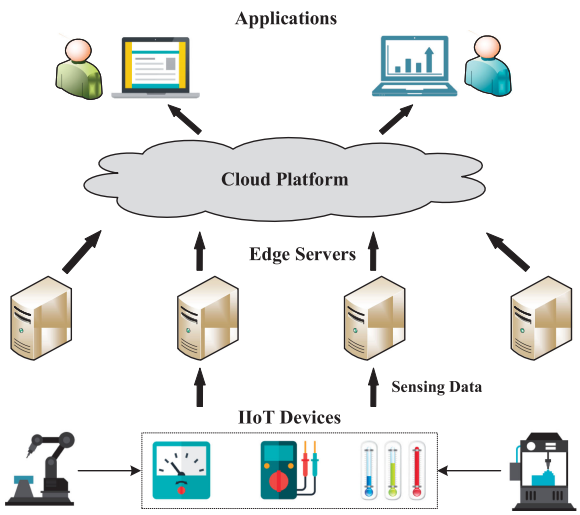


Fig. 1. Illustration of the edge-cloud architecture of IIoT.

Differential privacy, proposed by Dwork and Roth [10], has now become the de facto standard of privacy-preserving data publishing [11]. Taking a dataset as input, a differentially private mechanism outputs a randomized result, from which the adversary cannot accurately infer an individual's data. Because of the randomization, the utility of the data published by the edge server is lower than that of the original data. For the recipient of the published data, a large utility loss is not acceptable. Generally, the recipient does not fully trust the edge server. However, without accessing the original data, the recipient has no way to accurately evaluate the utility loss. To make things worse, the output of a differentially private mechanism may take values from an unbounded set. This gives the edge server plausible deniability. That is, the edge server can publish arbitrarily fabricated data without being caught.

In this article, we study how to incorporate the blockchain technique into the process of differentially private data publishing so as to guarantee the utility of published data. Specifically, we consider two types of data publishing, namely histogram publishing and anonymized data publishing. In the former case, the edge server is required to execute a series of count queries on the collected data. To meet the requirement of differential privacy, a Laplacian noise will be added to each query result. The utility loss is determined by the noise. To prevent the edge server from arbitrarily choosing the noise, we propose a protocol, which relies on the blockchain to send verified data to the recipient. The proposed protocol employs a verifiable shuffle scheme to decide the noise added in the published result. For anonymized data publishing, we adopt the DiffGen [12] algorithm, which outputs a set of anonymized data records obtained by generalizing original records. The algorithm also outputs a noisy frequency of each anonymized data record, which is obtained via a differentially private mechanism. Considering that the utility loss caused by generalization can be evaluated without the original data, we propose a data publishing protocol, which publishes the anonymized data records and the noisy frequencies separately. Under the assumption that the recipient must get the complete data, the proposed protocol can prevent the edge server and the recipient from lying about the utility loss.

The contributions of this article can be summarized as follows.

1) A blockchain-empowered differentially private data publishing framework is proposed. The proposed framework can help to realize privacy-preserving and fairness-guaranteed data sharing, which is demanded by IIoT applications.
2) A utility guaranteed histogram publishing protocol is proposed. By utilizing the verifiable shuffle scheme, the proposed protocol allows the nodes in the blockchain network to verify the noise added to the published result without leaking the noise.
3) A two-step protocol for anonymized data publishing is proposed. To reduce the burden of blockchain nodes, the proposed protocol delegates part of the utility verification work to the data recipient, yet still guarantees that the utility will be correctly evaluated.

The rest of this article is organized as follows. Section II introduces some basics of the tools used in our study. Section III describes the basic model of differentially private data publishing and presents the workflow of blockchain-aided publishing. Sections IV and V describe how to utilize the blockchain to guarantee data utility for histogram publishing and anonymized data publishing, respectively. Some illustrative numerical results are given in Section VI. Finally, Section VIII concludes this article.

## II. Preliminaries

### A. Differential Privacy

Consider the following data publishing scenario. A data requester wants to query a dataset $D$ that is managed by a data collector. The dataset is consisted of multiple individuals' records. To make sure that publishing results of the queries will not disclose much information about an individual's record, the data collector can apply a differentially private mechanism. A formal definition of differential privacy is given below [10].

*Definition 1 ($\varepsilon$-Differential privacy):* A randomized mechanism $\mathcal{M}$ is $\varepsilon$-differentially private if for any neighboring datasets $D_1$ and $D_2$ that differ on at most one record, and for any set of outputs $S \subseteq \text{Range}(\mathcal{M})$, there is

$$\Pr\left(\mathcal{M}\left(D_1\right) \in S\right) \leq e^{\varepsilon} \cdot \Pr\left(\mathcal{M}\left(D_2\right) \in S\right). \tag{1}$$

The *privacy budget* $\varepsilon$ controls the level of privacy protection that the mechanism $\mathcal{M}$ can guarantee. If the query $Q$ sent by the requester is numeric, then the collector can adopt the *Laplace mechanism* to achieve differential privacy. Formally, the output of the Laplace mechanism is given by $\tilde{Q}(D) = Q(D) + \sigma$, where $Q(D)$ is the real result of the query, $\sigma$ is a random number drawn from the Laplace distribution $\text{Lap}(0, \frac{\Delta Q}{\varepsilon})$. The probability density function of the distribution is given by $F_{\text{Lap}}(x \mid 0, \frac{\Delta Q}{\varepsilon}) \triangleq \frac{\varepsilon}{2\Delta Q} e^{-\frac{\varepsilon|x|}{\Delta Q}}$. The global sensitivity $\Delta Q$ in abovementioned equation measures the maximum difference between

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

query results on neighboring datasets. The Laplacian noise makes the output result inaccurate. The utility loss corresponding to $\tilde{Q}(D)$ can be measured by the error $|Q(D) - \tilde{Q}(D)|$. When sending the query, the requester can specify an error bound $\alpha$, which defines the maximum acceptable utility loss [13].

### B. Elliptic Curve ElGamal Scheme

The ElGamal public-key encryption scheme [14] is widely used in practice. In this article, we adopt the elliptic curve ElGamal scheme (EC-ElGamal) [15]. The public domain parameters of the scheme include a prime number $p$, the equation of an elliptic curve $E$ defined over a finite field $\mathbb{F}_p$, a base point $B$ in $E(\mathbb{F}_p)$, and its order $n$. Given the public parameters, the private key $k_{\text{pri}}$ is generated by selecting an integer from the interval $[1, \ldots, n-1]$ uniformly at random. The corresponding public key $k_{\text{pub}}$ is computed by $k_{\text{pub}} = k_{\text{pri}} B$.

To encrypt a message msg with the public key $k_{\text{pub}}$, one needs to represent msg as a point $\mathcal{P}_{\text{msg}}$ in $E(\mathbb{F}_p)$. The cipher text $C \triangleq (C_1, C_2)$ is consisted of two parts, where $C_1 = \nu B$ and $C2 = \mathcal{P}_{\text{msg}} + \nu k_{\text{pub}}$. The nonce $\nu$ is randomly selected from the interval $[1, \ldots, n-1]$. With the private key $k_{\text{pri}}$, one can decipher $C$ by first computing $\mathcal{P}_{\text{msg}} = C_2 - p_{\text{pri}} C1$ and then extracting msg from $\mathcal{P}_{\text{msg}}$. The EC-ElGamal scheme is additively homomorphic. That is, for any pair of messages $(m_1, m_2)$ and any pair of scalars $(a, b)$, there is $[\![am_1 + bm_2]\!]_{k_{\text{pub}}} = a[\![m_1]\!]_{k_{\text{pub}}} + b[\![m_2]\!]_{k_{\text{pub}}}$, where $[\![\cdot]\!]_{k_{\text{pub}}}$ denote the result of encrypting $m$ with the public key $k_{\text{pub}}$.

### C. Verifiable Shuffle

A verifiable shuffle scheme consists of three elements [16]: A public-key encryption scheme with a re-encryption algorithm $\mathcal{RP}$, a probabilistic polynomial-time algorithm for shuffling $\mathcal{S}$, and a prove system $(\mathcal{P}, \mathcal{V})$. Simply speaking, the shuffling algorithm $\mathcal{S}$ takes a list of ciphertexts $(c_1, \ldots, c_L)$ as input and outputs another list of ciphertext $(c'_1, \ldots, c'_L)$. The prover $\mathcal{P}$, namely the shuffler, is required to convince the verifier $\mathcal{V}$ that there is a permutation $\pi$ and a list of random nonces $(\beta_1, \ldots, \beta_L)$ such that $c'_i$ is obtained by re-encrypt the ciphertext $c_{\pi(i)}$ with the random nonce $\beta_i$ $(i = 1, \ldots, L)$. Meanwhile, the verifier learns nothing about $\pi$ and $(\beta_1, \ldots, \beta_L)$. Verifiable shuffle schemes are mainly used to construct mix-nets that can provide anonymity and unlinkability in communications. The EC-ElGamal encryption scheme has a re-encryption algorithm. Neff [17] proposed a proof protocol for the shuffle of ElGamal pairs. The protocol has the honest-verifier zero-knowledge property. And the protocol is unconditionally sound, in the sense that even if the prover has unlimited computational resources, it cannot forge a transcript with an interactive verifier. For more details of the protocol, see [17].

### D. Consortium Blockchain

The consortium blockchain is considered to be most applicable to scenarios where a group of organizations need to collaborate but do not fully trust each other [18]. A consortium blockchain uses a set of preselected nodes to run the consensus protocol, and only authenticated entities can read/write transactions. The basic transaction flow in a consortium blockchain can be briefly described as follows. A user submits a transaction via a registered client. The client is connected to a node in the blockchain network. The node verifies the signature of the transaction and adds it to a transaction pool, if the pool is not full. A verification node draws transactions from the transaction pool and packs them into a block. The block is then broadcasted to other verification nodes, and the consensus procedure begins. Each verification node executes every transaction within the block and verifies the execution result. Suppose that the verification nodes adopt the practical Byzantine fault tolerance (PBFT) consensus algorithm [19]. When more than two thirds of the verification nodes have got the same result, a consensus is achieved. Then, the block will be appended to the ledger maintained by each node.

## III. System Model

### A. Noninteractive Differentially Private Data Publishing

The privacy-preserving data publishing scenario studied in this article can be described as follows. An edge server $S$, which plays the role of *data collector*, collects sensing data from $N$ IIoT devices (e.g., sensors, actuators, and meters). The edge server and the devices belong to the same organization. Each device is associated with an individual whom we refer to as *data contributor*. The sensing data from a contributor are represented as a record consisting of multiple attributes. After conducting some preprocessing, the data collector uploads a dataset to the cloud platform where the data are analyzed to support certain applications. Denote the dataset as $D \triangleq \{r_1, r_2, \ldots, r_N\}$, where $r_i$ $(i = 1, 2, \ldots, N)$ is the data record from the $i$th data contributor. Let $\mathcal{A} \triangleq \{A_1, \ldots, A_M\}$ denote the set of all attributes. The value of attribute $A_j$ in record $r_i$ is denote as $r_i.A_j$.

If some user from another organization is interested in the data, he can send a query $Q$ to the data collector. We refer to the user as *data requester*. To protect the privacy of data contributors, the data collector constructs a differentially private mechanism $\mathcal{M}_Q$ and applies it to the data set $D$. The output of the mechanism, denoted as $\tilde{D}$, is then published to the requester. Generally, the requester is required to pay a certain amount of rewards to the collector. Let $U_{D,\tilde{D}}$ denote the utility loss corresponding to $\tilde{D}$. The requester can refuse to pay the rewards if $U_{D,\tilde{D}}$ is larger than a threshold.

There are two typical settings in differentially private data publishing [13], namely *interactive* publishing and *noninteractive* publishing. In this article, we focus on the second setting. Specifically, we consider the following two scenarios.

*1) Histogram Publishing:* If the data requester is interested in a specific attribute $A_j \in \mathcal{A}$, he can asks to the data collector to publish a histogram $\mathbf{h}_j \triangleq \{f_1, \ldots, f_K\}$ with $K$ denoting the number of bins and $f_k$ $(k = 1, \ldots, K)$ denoting the frequency of the $k$th bin $\mathcal{B}_k$. In such a case, the query $Q$ sent by the requester can be decomposed into $K$ count queries $\{Q_k\}_{k=1,\ldots,K}$. In order to hide the real frequency $f_k \triangleq Q_k(D)$, the data collector applies a Laplace mechanism. The noisy frequency $\tilde{f}_k$ published

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

to the requester is computed by $\tilde{f}_k = f_k + \sigma_k$. The random noise $\sigma_k$ is drawn from the Laplace distribution Lap$(0, \frac{1}{\epsilon_k})$, where $\epsilon_k$ denotes the privacy budget allocated to query $Q_k$. According to the sequential composition property of differential privacy [13], publishing the noisy histogram $\tilde{\mathbf{h}}_j \triangleq (\tilde{f}_1, \ldots, \tilde{f}_K)$ satisfies $(\sum_{k=1}^{K} \epsilon_k)$-difffernetial privacy. With regard to the data utility, the published histogram $\tilde{\mathbf{h}}_j$ should satisfy

$$\max_{k=1,\ldots,K} \left| f_k - \tilde{f}_k \right| \leq \alpha \tag{2}$$

where $\alpha$ is the error bound specified by the data requester.

*2) Anonymized Dataset Publishing:* In order to conduct a complex analysis of the data, the data requester may ask the data collector to publish a dataset that has the same format with the original one. In this article, we adopt the algorithm *DiffGen* proposed in [12]. It is assumed that each data record in the original dataset is consisted of three types of attributes, namely an *explicit identifier*, a *class attribute,* and multiple *predictor attributes*. The explicit identifier, which explicitly identifies an individual will be removed from the published dataset. Values of predictor attributes can be used to predict the value of the class attribute. The goal of the data requester is to build a classifier by using the published data.

Given a dataset $D$, DiffGen generates an anonymized dataset $\tilde{D}$ via two steps. In the first step, the algorithm transforms $D$ into a set of equivalence groups by generalizing values of predictor attributes. Without loss of generality, we assume that $A_1$ is the identifier, $A_M$ is the class attribute, and $A_2, \ldots, A_{M-1}$ are the predicator attributes. Each predictor attribute $A_j$ is associated with a taxonomy tree $T_{A_j}$, according to which the values of the attribute are generalized. An equivalence group can be represented by $(\tilde{r}_l, f_l)$, where $\tilde{r}_l$ is the anonymized record, and $f_l$ denotes the count of original data records generalized to $\tilde{r}_l$. In the second step, the algorithm adopts the Laplace mechanism to add random noise to each $f_l$ so as to guarantee $\epsilon$-differential privacy. Let $\tilde{f}_l$ denote the noisy count. The output dataset $\tilde{D}$ can be written as $\{(\tilde{r}_l, \tilde{f}_l)\}_{l=1,\ldots,L}$.

In DiffGen, both the generalization operation and the use of Laplace mechanism cause a decline in data utility. Given $D$ and $\tilde{D}$, We define the utility loss as $U_{D,\tilde{D}} = (U_{\text{Gen}}(\tilde{D}), E_{\text{Lap}}(D, \tilde{D}))$, where $U_{\text{Gen}}(\cdot)$ measures the utility loss caused by generalization, and $E_{\text{Lap}}(\cdot, \cdot)$ measures the error caused by the Laplacian noise. Given an anonymized record $\tilde{r}$, the utility loss is defined as $U_{\text{record}}(\tilde{r}) = \sum_{m=2}^{M-1} \omega_m U_{\text{attr}}(\tilde{r}.A_m)$, where $\omega_m \in [0, 1]$ denotes the weight associated with $A_m$, and $U_{\text{attr}}(\tilde{r}.A_m)$ is defined as

$$U_{\text{attr}}(\tilde{r}.A_m) = \frac{\text{size}(\tilde{r}.A_m)}{\text{size}(A_m)}. \tag{3}$$

If $A_m$ is categorical, then size$(A_m)$ denotes the number of leaf nodes in the taxonymy tree $T_{A_j}$, and size$(\tilde{r}.A_m)$ dentoes the number of leaf nodes in the subtree rooted at the node whose value is $\tilde{r}.A_m$. If $A_m$ is numberical, then size$(A_m)$ is defined as $\max_{r \in D}\{r.A_m\} - \min_{r \in D}\{r.A_m\}$, and size$(\tilde{r}.A_m)$ denotes the length of $\tilde{r}.A_m$, which corresponds to an interval contained in $[\min_{r \in D}\{r.A_m\}, \max_{r \in D}\{r.A_m\}]$. The utility loss $U_{\text{Gen}}(\tilde{D})$ is computed by

$$U_{\text{Gen}}\left(\tilde{D}\right) = \frac{1}{L} \sum_{l=1}^{L} U_{\text{record}}(\tilde{r}). \tag{4}$$

The error $E_{\text{Lap}}(D, \tilde{D})$ is defined as $\max_{l=1,\ldots,L} |f_l - \tilde{f}_l|$.

## B. Incorporate Blockchain Into Data Publishing

To assist users to discover data that may be helpful to their applications, and to guarantee the audibility of the data publishing process, we propose a consortium blockchain-based framework where the data exchanged between the collector and the requester are stored in a tamper-resistant manner. Several organizations that share the same IIoT infrastructure forms a consortium. Each organization owns a number of edge servers. After registering at a certificate authority (CA), an edge server can join the *blockchain network* and becomes a *node*. There are two types of nodes in the network. The *verification nodes* jointly maintain a public ledger $\mathcal{L}$. They are responsible for verifying transactions, generating blocks, and running the consensus algorithm. The nonverificaiton nodes do not participate in bookkeeping transactions. They are mainly responsible for transmitting transactions and broadcasting blocks. A *client*, which has registered at the CA can send transactions to the blockchain network.

In the data publishing scenario described in abovementioned section, the data collector and the data requester can register as clients. During the registration, the collector gets a pair of EC-Elgamal keys $(P_C, S_C)$ from the CA. The requester also gets $(P_R, S_R)$. These keys will be used in the data sharing process. The two clients interact with each other by invoking a smart contract $\mathcal{SC}$ that has been deployed on the blockchain. The transactions generated by the smart contract will be recorded in the ledger $\mathcal{L}$. The basic workflow of blockchain-based data publishing consists of the following steps.

1) *Publishing the Metadata:* Once obtaining the data set $D$, the data collector invokes the smart contract to publish some metadata about $D$. The metadata $\boldsymbol{\omega}_D$ includes at least the following information: The hash value of the data set hash$(D)$, the number of data records $N$, the set of attributes $\mathcal{A}$, the maximum privacy budget $\varepsilon_{\text{max}}$, which is agreed by all the related data contributors, and the probability quantum parameter $\delta$, which will be used to generate Laplacian numbers. The transaction generated by the smart contract is denoted as $TX_{\text{meta}}$.
2) *Retrieving the Metadata:* The data requester invokes the smart contract to retrieve the transaction $TX_{\text{meta}}$ and gets the metadata $\boldsymbol{\omega}_D$. The corresponding transaction is denoted as $TX_{\text{getm}}$.
3) *Issuing the Query:* The requester who is interested in $D$ invokes the smart contract to issue a query $Q$. The requester also specifies the maximum acceptable utility loss $\alpha$ and the reward $\rho$ he would like to pay for the data. The corresponding transaction is denoted as $TX_{\text{query}}$. To prevent the requester from repudiating the payment, it is

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

required that the requester deposits the reward to the smart contract's account.

4) *Publishing the Result:* After receiving the query, the collector applies the differentially private algorithm $\mathcal{M}_Q$ to get the result $\tilde{D}$. Then, the collector invokes the smart contract to publish the result. The corresponding transaction is denoted as $TX_{\text{result}}$.
5) *Transferring the Reward:* The transaction $TX_{\text{result}}$ will not be appended to the ledger $\mathcal{L}$ unless it has been verified by the verification nodes. During the verification process, the verification nodes will check if the utility loss corresponding to $\tilde{D}$ meets the requester's requirement. After the transaction $TX_{\text{result}}$ is appended to the ledger, the reward deposited in the smart contract will be transferred to the collector's account automatically.
6) *Retrieving the Result:* After $TX_{\text{result}}$ is appended to the ledger, the requester invokes the smart contract to retrieve the result published by the collector. The corresponding transaction is denoted as $TX_{\text{getr}}$.

## IV. Utility-Guaranteed Differentially Private Histogram Publishing

When the data collector is required to publish a histogram to the data requester, the utility loss of the noisy result $\tilde{\mathbf{h}}_j$ should satisfy the condition defined in (2). Both the requester and the verification nodes in the blockchain network are not allowed to know the real frequencies. Moreover, the verification nodes are not allowed to know the noisy frequencies, since they do not pay for the data. In this section, we propose a data publishing protocol, which relies on the verification nodes to guarantee the utility. And it is ensured that the verification nodes learns no useful information from the data exchanged between the collector and the requester.

### A. Blockchain-Assited Histogram Publishing

As described in Section III-B, the data collector and the data requester interact with each other by sending transactions to the blockchain network. In order to get a histogram, the requester calls the smart contract to send a transaction $TX_{\text{query}} \triangleq (\{Q_k\}_{k=1,\ldots,K}, \{\varepsilon_k\}_{k=1,\ldots,K}, \alpha, \rho)$ to the blockchain network, where $\varepsilon_k$ is the privacy budget that the requester allocates to the query $Q_k$. During the consensus process, the verification nodes will check if the total privacy budget $\sum_{i=1}^{K} \varepsilon_k$ exceeds the maximum privacy budget $\varepsilon_{\max}$ that the collector specified in the transaction $TX_{\text{meata}}$. Only if $\sum_{i=1}^{K} \varepsilon_k \leq \varepsilon_{\max}$, the transaction $TX_{\text{query}}$ can be appended to the ledger $\mathcal{L}$. The collector invokes the smart contract to retrieves the privacy budgets $\{\varepsilon_k\}_{k=1,\ldots,K}$ and the error bound $\alpha$. For each $k \in \{1, \ldots, K\}$, the noisy result $\tilde{f}_k$ is published via the following way.

At first, the collector, the requester and each verification node generates a list of Laplace random numbers, respectively. Inspired by Froelicher *et al.* [7], we design a Monte Carlo like method to generate the random numbers. As described in Algorithm 1, a parameter $\delta$ is specified to quantize the probability. Given the input parameters $\alpha$, $\delta$, and $\varepsilon_k$, the output of Algorithm 1 is deterministic. That is to say, if all the parties mentioned previously execute the algorithm correctly, they will get the same list of numbers. Let $\mathcal{N}_k$ denote the list generated for $\varepsilon_k$. The length of the list is denoted as $N_K$. As long as the noise $\sigma_k$ that the collector adds to the real result $f_k$ is picked from the list, the noisy result $\tilde{f}_k$ will satisfy both the differential privacy requirement and the utility requirement. The random number selected by the collector should be hidden from the requester and the verification nodes. Meanwhile, the verification nodes should assure the requester that the result $\tilde{f}_k$ is computed with a number from the list $\mathcal{N}_k$.

**Algorithm 1:** Generate a List of Random Numbers Following Laplace Distribution.

**Input:** The error bound $\alpha$, the probability quantum parameter $\delta$, and the privacy budget $\varepsilon_k$
**Output:** A list of random numbers $\mathcal{N}$

```
N ← ∅
Z_α ← {z | z ∈ ℤ and |z| ≤ α}
z ← ⌈−α⌉ // ⌈x⌉ denotes the minimum integer which is no smaller than x
while z ≤ ⌊α⌋ do// ⌊x⌋ denotes the maximum integer which is no larger than x
    p ← 0
    while p ≤ F_Lap(z; 1/ε_k) do
        add z to the end of the list N
        p ← p + δ
    end while
    z ← z + 1
end while
Return N
```

After generating the list $\mathcal{N}_k$, the requester reorders the numbers in $\mathcal{N}_k$ and gets a new list $\mathcal{N}'_k$. Then, the requester invokes the smart contract to send the new list to the blockchain network. Denote the corresponding transaction as $TX_{\text{LapList}}$. During the consensus process, each verification node first utilizes the list $\mathcal{N}_k$ generated by itself to check if $\mathcal{N}'_k$ is valid. After that, one of the verification nodes constructs a joint public key $P_{CR} \triangleq P_C + P_R$, and encrypts each number in the list $\mathcal{N}'_k$ with this public key. Let $z_i \triangleq (\beta_i B, y_i + \beta_i P_{CR})$ denote the cipher text corresponding to the $i$th number $y_i$ in $\mathcal{N}'_k$, where $\beta_i$ is random nonce chosen by the node. The list of ciphertexts $(z_1, \ldots, z_{N_K})$ is sent to the collector and other verification nodes. In addition, the list of random nonces $(\beta_1, \ldots, \beta_{N_K})$ is sent to the collector.

The collector applies a verifiable shuffle algorithm [17] to the received list $(z_1, \ldots, z_{N_K})$. The algorithm outputs a list of ciphertexts $(z'_1, \ldots, z'_{N_K})$. As described in Section II-C, each ciphertext $z'_i$ is a re-encryption of some element $z_{\pi(i)}$ in the input list. The permutation $\pi$ is only known to the collector. The ciphertext $z'_i$ can be written as $z'_i = ((\gamma_i + \beta_{\pi(i)})B, y_{\pi(i)} + (\gamma_i + \beta_{\pi(i)})P_{CR})$, where $\gamma_i$ is the random nonce used to re-encrypt $z_{\pi(i)}$ and is only known to the collector. It should be noted that since the collector has got $(\beta_1, \ldots, \beta_{N_k})$ from the verification node, the plaintext corresponding to $z'_i$, namely $y_{\pi(i)}$, is known to the collector. In the original verifiable

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

shuffle protocol, the shuffler is not supposed to know the correspondence between ciphertext and plaintext. While in the data publishing scenario studied in this article, it is the collector's job to decide the noise added to the result, hence, there is no need to hide $y_{\pi(i)}$ from the collector.

After generating the shuffled list, the collector first computes $\tilde{f}_k = f_k + y_{\pi(i)}$, and then encrypts the real frequency $f_k$ with the joint public key $P_{CR}$. The encryption result is $[\![f_k]\!]_{P_{CR}} \triangleq (\eta_k B, f_k + \eta_k P_{CR})$, where the random nonce $\eta_k$ is only known to the collector. After that, the collector invokes the smart contract to send the following transaction to the blockchain network:

$$TX_{res,k} = \left([\![f_k]\!]_{P_{CR}}, \text{hash}(\eta_k) \left(z'_1, \ldots, z'_{N_k}\right), \mu_{k,1}, \text{hash}\left(\tilde{f}_k\right)\right) \quad (5)$$

where $\mu_{k,1} = \beta_{\pi(1)} + \gamma_1 + \eta_k$. It is defined that the Laplace random number $y_{\pi(1)}$ corresponding to the first cipher text $z'_1$ is chosen as the noise added to the result. Hence the collector only include $\mu_{k,1}$ in the transaction. Later the requester can use this number to decipher the noisy result.

The transaction $TX_{res,k}$ will not be written in the ledger unless the verification nodes reach a consensus on the validity of the transaction. During the consensus process, each verification node runs the proof protocol corresponding to the verifiable shuffle algorithm to verify if the list $(z'_1, \ldots, z'_{N_k})$ is correctly generated from the list $(z_1, \ldots, z_{N_k})$. If the two lists do not match, $TX_{res,k}$ will be treated as an invalid transaction. After $TX_{res,k}$ is written in the ledger, the requester can invoke the smart contract to retrieve the data included in the transaction. By using the homomorphic property, the requester first computes the encrypted noisy result as

$$\begin{aligned}\left[\!\left[\tilde{f}_k\right]\!\right]_{P_{CR}} &= [\![f_k]\!]_{P_{CR}} + z'_1 \\ &= \left(\left(\eta_k + \gamma_1 + \beta_{\pi(1)}\right) B \right. \\ &\quad \left. f_k + y_{\pi(1)} + \left(\eta_k + \gamma_1 + \beta_{\pi(1)}\right) P_{CR}\right). \end{aligned} \quad (6)$$

Then, by using $\mu_{k,1}$ from the transaction $TX_{res,k}$, the requester can decipher $[\![\tilde{f}_k]\!]_{P_{CR}}$ to get the result $\tilde{f}_k$. Furthermore, the requester can use the hash value hash$(\tilde{f}_k)$ to verify if the decryption is correct. The whole publishing procedure of $\tilde{f}_k$ is illustrated in Fig. 2.

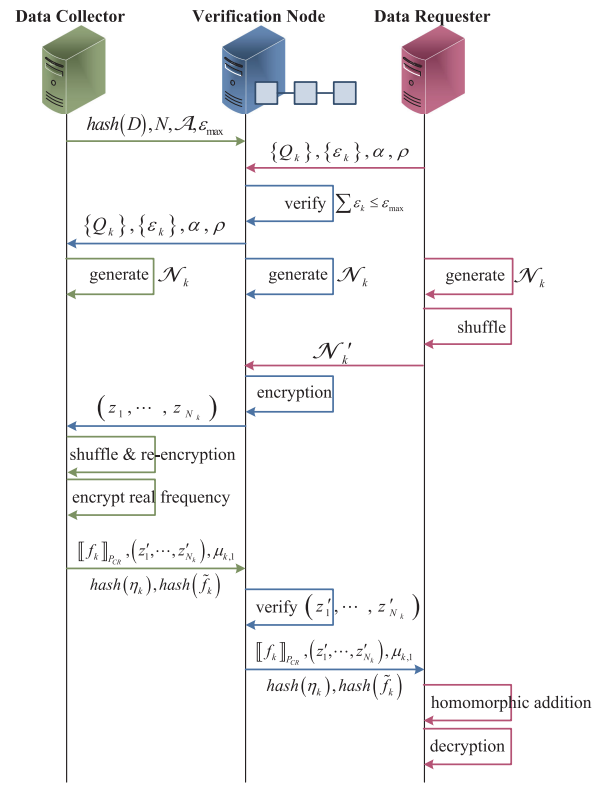


Fig. 2. Sequence diagram of the proposed protocol for publishing a noisy frequency.

## B. Anaylis

In differentially histogram publishing, the Laplacian noise added to the real frequency determines not only the level of privacy protection achieved by the publishing mechanism, but also the utility of the noisy frequency obtained by the requester. A dishonest collector may attempt to add arbitrary noise to the real frequency. The data publishing protocol described previously can guarantee that the noise added by the collector coincides with the privacy budget agreed by the data contributors and the error bound specified by the requester. Specifically, the noise chosen by the collector must be verified by the verification nodes. Given the public parameters, namely the privacy budget $\varepsilon_k$, the probability quantum parameter $\delta$, and the error bound $\alpha$, each verification node can decide the proper "candidates" for the Laplacian noise. The collector is required to prove to the verification nodes that the chosen noise does come from the candidate list. Moreover, the noisy frequency is not directly sent from the collector. Instead, the noisy frequency is computed by the requester himself. As a result, the collector is not able to add an arbitrary noise. Theoretically, the collector can fabricate the value of $\mu_{k,1}$ if it doesn't want the requester to get the result. However, if $\mu_{k,1}$ is not consistent with the random nonces used in the encryption of $f_k$ and $y_{\pi(1)}$, the decryption of $\tilde{f}_k$ will fail. Knowing that the requester will easily discover its misbehaving, a rational collector has no motive to fabricate the value of $\mu_{k,1}$.

A dishonest requester may attempt to get the real frequency $f_k$. While, the real frequency is encrypted with the joint public key $P_{CR}$, and the requester cannot decipher the data without the help of the collector. As defined in (5), the collector sends the summary of three random nonces to the requester. The requester cannot infer the random nonce $\eta_k$ from $\mu_{k,1}$, thus, he cannot use $\eta_k$ to decipher $[\![\tilde{f}_k]\!]_{P_{CR}}$. The requester may try to reveal the Laplacian noise $y_{\pi(1)}$ and use it to infer the real frequency. However, the permutation $\pi$ is only known to the collector, and the requester cannot infer $\gamma_i$ or $\beta_{\pi(i)}$ from $\mu_{k,1}$, thus, he cannot decipher $z'_1$.

A dishonest verification node may attempt to learn information about the frequency from the transaction sent by the collector. However, the real frequency is encrypted with the joint public key. Without the help of the collector and the

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

requester, the verification node cannot decipher the data. As for the Laplacian noise, though the verification node knows the plain value corresponding to each element in $(z_1, \ldots, z_{N_k})$, the shuffle algorithm used the by the collector makes sure that the verification node cannot infer the plain value corresponding to $z'_1$. Therefore, even if the verification node colludes with the requester, it cannot help the requester to infer the real frequency.

One premise for the requester to get the correct noisy frequency is that the encrypted real frequency $[\![\tilde{f}_k]\!]_{P_{CR}}$ provided by the collector is consistent with the original dataset $D$. In our work, the collector is required to publish the hash value $\text{hash}(D)$ before anyone sends queries about the data. During the data publishing process, the encrypted real frequency $[\![\tilde{f}_k]\!]_{P_{CR}}$ is written in the blockchain's ledger and can no longer be modified. If the requester has doubt about the authenticity of the real frequency, he can resort to an authoritative third party. The third party will ask the collector to hand over the dataset, which should be consistent with the hash value published on chain, and the random nonce $\eta_k$ used to encrypt to the real frequency. Then, by executing the query $Q_k$ on the dataset and encrypting the query result, the third party can determine if the collector has provided the right information. Knowing that its dishonest behavior can be discovered, a rational collector has no motive to fabricate the real frequency.

## V. Utility-Guaranteed Anonymized Dataset Publishing

When the collector is required to publish an anonymized dataset $\tilde{D}$ to the requester, the verification nodes in the blockchain network need to ensure that the utility of the anonymized data satisfies the following condition: $U_{\text{Gen}}(\tilde{D}) \leq \alpha_{\text{Gen}}$ and $E_{\text{Lap}}(D, \tilde{D}) \leq \alpha_{\text{Lap}}$, where $\alpha_{\text{Gen}}$ and $\alpha_{\text{Lap}}$ are thresholds specified by the requester. Since the verification nodes are not allowed to see $D$ or $\tilde{D}$, to enable the verification of $U_{\text{Gen}}(\tilde{D})$, the collector can encrypt the published data. That is, for each predictor attribute $A_m$, the collector constructs an encrypted version of the taxonomy tree $T_{A_m}$ by encrypting each node value with the requester's public key $P_R$. Based on the encrypted taxonomy tree $[\![T_{A_m}]\!]_{P_R}$, the collector can generate an encrypted version of $\tilde{D}$ by changing each value $\tilde{r}_l.A_m$ with the corresponding cipher text in $[\![T_{A_m}]\!]_{P_R}$. Upon receiving the cipher texts from the collector, the verification node can utilize the homomorphic property to compute the encrypted value of $U_{\text{Gen}}(\tilde{D})$. Then, by applying a secure comparison protocol, the verification node is able to determine if $U_{\text{Gen}}(\tilde{D}) \leq \alpha_{\text{Gen}}$ holds. However, the approach described above involves a lot of calculations on ciphertext. When the number of records or the number of attributes is large, the verification process may take a very long time. In this section, we propose a more efficient way to guarantee the utility of the published data.

### A. Two-Step Data Publishing Protocol

It is assumed that the goal of the data requester is to build a classifier with the anonymized dataset $\tilde{D}$. Each anonymized record $\tilde{r}_l$ consists of $M-2$ predictor attributes and one class attribute. If only the values of predictor attributes or the values of class attribute are published, the requester has no way to learn the relationships between predictor attributes and the class attribute. Moreover, as introduced in Section III-A2, the evaluation of the utility loss caused by generalization only involves the predictor attributes. With this in mind, we propose the following data publishing protocol.

After generating the anonymized dataset $\tilde{D}$, the collector first divides each record $\tilde{r}_l$ into two parts, namely $\tilde{r}_{l,\text{pre}}$ and $\tilde{r}_{l,\text{cls}}$. The first part contains only the values of predictor attributes, and the second part contains only the value of the class attribute. Define $\tilde{D}_{\text{pre}} = \{\tilde{r}_{1,\text{pre}}, \ldots, \tilde{r}_{L,\text{pre}}\}$ and $\tilde{D}_{\text{cls}} = \{\tilde{r}_{1,\text{cls}}, \ldots, \tilde{r}_{L,\text{cls}}\}$. The collector then encrypts $\tilde{D}_{\text{pre}}$ with the requester's publik key $P_R$, and sends the following transaction:

$$TX_{\text{pre}} = \left( \left[\!\left[ \tilde{D}_{\text{pre}} \right]\!\right]_{P_R}, \text{hash}\left(\tilde{D}_{\text{pre}}\right) \right.$$
$$\left. \{[\![T_{A_m}]\!]_{P_R}\}_{m=2}^{M-1}, \{\text{hash}\left(T_{A_m}\right)\}_{m=2}^{M-1} \right). \quad (7)$$

Each taxonomy tree $T_{A_m}$ is encrypted with the requester's public key. Later the requester can use the taxonomy trees to evaluate the utility loss $U_{\text{Gen}}(\tilde{D})$. During the consensus process, the verification nodes just conduct an ordinary verification of the transaction $TX_{\text{pre}}$.

After retrieving the transaction from the ledger, the requester can decipher the data with his privacy key. The hash values included in $TX_{\text{pre}}$ are utilized to check the integrity of the decryption result. Then, the requester can compute the utility loss $U_{\text{Gen}}(\tilde{D})$. If the loss is satisfying, the requester needs to send an acknowledgement transaction $TX_{\text{ack}}$. Once seeing the acknowledgment, the collector can publish the values of class attribute and the noisy frequencies $\{\tilde{f}_l\}$. The frequencies are published via the same way described in abovementioned section. The corresponding transaction $TX_{\text{cls}}$ consists of $[\![\tilde{D}_{\text{cls}}]\!]_{P_R}$, $\text{hash}(\tilde{D}_{\text{cls}})$, $\{[\![f_l]\!]_{P_{CR}}\}_{l=1}^{L}$, $\{[\![\eta_l]\!]_{P_{CR}}\}_{l=1}^{L}$, $\{(z'_1, \ldots, z'_{N_l})\}_{l=1}^{L}$, $\{\mu_{l,1}\}_{l=1}^{L}$, and $\{\text{hash}(\tilde{f}_l)\}_{l=1}^{L}$. During the consensus process, the verification nodes will check the validity of each shuffled list $(z'_1, \ldots, z'_{N_l})$. After retrieving $TX_{\text{cls}}$ from the blockchain, the requester first uses his private key to decipher $[\![\tilde{D}_{\text{cls}}]\!]_{P_R}$. Then, he can use the same method described in Section IV-A to get the noisy frequencies $\{\tilde{f}_l\}$.

### B. Analysis

According to the work flow described in Section III-B, the requester is required to make a deposit on the blockchain when issuing the query. We define that after the requester gets the first part of data $\tilde{D}_{\text{pre}}$ and sends the acknowledgement transaction $TX_{\text{ack}}$, a part of the deposit will be automatically transferred to the collector. Based on the assumption that the requester must obtain the complete data, which include the predictor attributes, the class attribute, and the frequency of each record, to build a classifier, a rational requester will not lie about the utility loss $U_{\text{Gen}}(\tilde{D})$. Suppose that the collector is honest, in the sense that the utility loss of the published data $\tilde{D}_{\text{pre}}$ is no larger than the threshold $\alpha_{\text{Gen}}$. If the requester refuse to acknowledge this fact, then he will not send the acknowledgement transaction and the collector will not publish the rest of the data. As a result, the requester cannot get the complete data.

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

A dishonest collector may try to publish a dataset whose utility does not satisfy the requester's requirement. As discussed in Section IV-B, the utility loss of the noisy frequency $\tilde{f}_l$ can be guaranteed with the help of the verification nodes. Suppose that the collector tries to publish a dataset $\tilde{D}_{\text{pre}}$ whose utility loss is larger than $\alpha_{\text{Gen}}$. Since the taxonomy trees are usually common knowledge, which cannot be defined arbitrarily, the requester is able to find out that $\tilde{D}_{\text{pre}}$ is unqualified. Consequently, the requester will not send the acknowledgement, and the collector cannot get the payment. That is to say, the collector has no motive to publish an unqualified dataset.

## VI. Simulation

In abovementioned sections, we have elaborately described how to ensure the utility of the data published by the collector satisfies the requester's requirement. Other than performing the ordinary validations of transactions, the verification nodes are responsible for verifying the noise added in published frequencies. Because of the additional verification, it will take a longer time for a transaction to be written in the ledger. In this section, we evaluate the efficiency of the proposed protocols by conducting several simulations.

We use the open-source consortium blockchain platform FISCO BCOS[1] to conduct the simulations. A consortium blockchain consisting of four nodes is deployed on a desktop workstation. We follow the instructions in the official documentation to configure the chain. The PBFT consensus algorithm is adopted. In the protocol proposed for anonymized data publishing, the verification nodes just verify the Laplacian noise added in the published frequencies, which is similar to the histogram publishing case. Hence, we only simulate the histogram publishing scenario. To conduct the simulations, we prepare a synthetic dataset, which consists of 1000 data records. Each data record contains only one numerical attribute which takes value from $\{1, \ldots, 10\}$. Two clients are created to mimic the collector and the requester. The query of the requester consists of 10 subqueries $Q_k$ ($k = 1, \ldots, 10$), where $Q_k$ counts the number of records whose value equals to $k$. We allocate the same privacy budget $\varepsilon_k$ to each $Q_k$.

To evaluate the performance of the proposed protocol, we first simulate a basic data publishing protocol, which does not involve the verification of data utility. For each query $Q_k$, the collector computes the noisy frequency and encrypts it with the requester's public key. The cipher text is published to the requester who then performs a decryption to get the noisy frequency. We measure the total run time of on-chain operations and off-chain operations, respectively. The on-chain operations mainly refer to the execution of the transactions $TX_{\text{query}}$, $TX_{\text{result}}$ and $TX_{\text{getr}}$. The off-chain operations include data encryption and decryption. As for the proposed data publishing protocol, the off-chain operations mainly include the shuffling and the re-encryption of Laplacian numbers, the encryption of real frequencies, the homomorphic addition, and the decryption of noisy frequencies. Based on the type of the transaction, the on-chain operations may include the verification of the shuffled Laplacian numbers. We measure the on-chain/off-chain run time, and compare the result with that of the basic protocol. For the proposed protocol, both the on-chain runtime and off-chain runtime are related to the count of Laplacian numbers generated by Algorithm 1, and the count is determined by the error bound $\alpha$, the probability quantum parameter $\delta$ and the privacy budget $\varepsilon_k$. Considering this, we test multiple settings of these input parameters. Given a group of $\alpha$, $\delta$, and $\varepsilon_k$, we run the abovementioned simulations 10 times and compute the average run time.

[1] [Online]. Available: http://www.fisco-bcos.org/

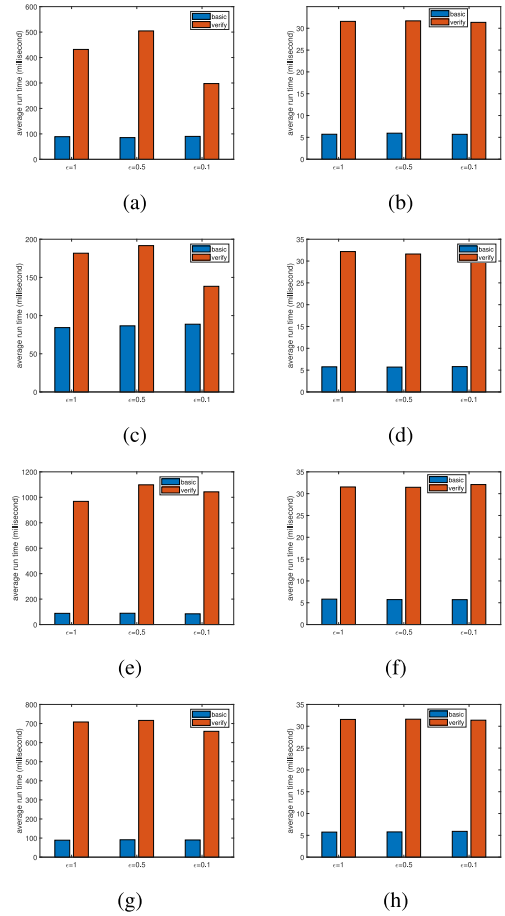


Fig. 3. Simulation results. The blue bar in each figure represents the run time of the basic publishing protocol, and the red bar represents the run time of the proposed protocol. (a) on-chain, $\alpha = 5$, $\delta = 0.01$. (b) off-chain, $\alpha = 5$, $\delta = 0.01$. (c) on-chain, $\alpha = 5$, $\delta = 0.05$. (d) off-chain, $\alpha = 5$, $\delta = 0.05$. (e) on-chain, $\alpha = 50$, $\delta = 0.01$. (f) off-chain, $\alpha = 50$, $\delta = 0.01$. (g) on-chain, $\alpha = 50$, $\delta = 0.05$. (h) off-chain, $\alpha = 50$, $\delta = 0.05$.

The simulation results are shown in Fig. 3. As expected, the proposed data publishing protocol is less efficient than the basic protocol. The basic protocol and the proposed protocol both involve data encryption and decryption operations, which are conducted off chain. While in the proposed protocol, the data collector not only encrypts the frequency, but also re-encrypts a list of Laplacian numbers. The on-chain run time of the proposed protocol is much longer than that of the basic protocol. This is mainly because that the verifiable shuffle protocol utilized in our work involves multiround interactions between the prover (i.e., the collector) and the verifier (i.e., the verification nodes).

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

Moreover, the shuffled list $(z'_1, \ldots, z'_{N_k})$ generated by the collector needs to be verified by each verification node so that they can achieve a consensus. In other words, the collector needs to interact with each verification node. These interactions are considered to be on-chain operations in our simulations.

For the proposed protocol, by comparing the results obtained under different settings of $\alpha$ and $\delta$, we can see that the on-chain run time declines significantly as $\delta$ becomes larger (which indicates a shorter list of Laplacian numbers), while the off-chain run time changes little. For example, when $\alpha = 5$, $\epsilon = 0.5$, and $\delta = 0.01$, the average run time of on-chain operations is about 500 milliseconds. While when $\delta = 0.05$, the average run time drops below 200 ms. This again demonstrates that most on-chain run time is spent on the verification of the shuffling. In future work, we will explore the noninteractive verifiable shuffle protocols so as to shorten the verification time.

## VII. Related Work

### A. Verifiable Differential Privacy

In the study of differentially private data publishing, it is usually assumed that the data collector is trusted by data contributors. Considering that in practice, the collector may behave dishonestly, researchers began to study the problem of verifiable differential privacy. Early studies [20], [21] mainly focus on how to develop practical tools that can provide formal guarantees of differential privacy. Narayan *et al.* [22] proposed a system called VerDP, which certifies not only the privacy but also the integrity of query results. In the scenario considered by VerDP, the private data are collected by a collector and then provided to an analyst to execute the query. The commitment to the private data is used to verify if the analyst actually uses the data to generate the query result. In the scenario studied in this article, the edge server that collects data from sensing devices plays the role of data collector and data analyst simultaneously, hence, the VerDP approach is not applicable.

### B. Integration of Blockchain and Differential Privacy

Though the privacy issues in blockchain systems have been widely investigated, we have only seen a few studies that explore the integration of blockchain and differential privacy. Zhao *et al.* [23] proposed to use blockchain to record the type of each query, the noisy response to each query, and the privacy budget allocated to each query in differentially private data publishing. Froelicher *et al.* [7] proposed a system called *UnLynx* that can support privacy-preserving data sharing. A distributed ledger is used to record the access rights, the query history and global parameters. In UnLynx, data providers execute queries on their data and send encrypted results to a server. The server performs a differentially private aggregation of the results. A verifiable shuffle protocol is employed to determine the noise added to the aggregated result. We also utilize the verifiable shuffle protocol in our work, while the data sharing model is different. As described in Section III, the collector directly executes the query on the data collected from sensing devices, and nodes in the blockchain network are responsible for verifying the noise.

## VIII. Conclusion

IIoT applications now have an increasing demand for privacy-preserving data sharing. In this article, we presented a blockchain-based framework for differentially private data publishing. Two protocols were proposed for histogram publishing and anonymized data publishing, respectively. By recording the interactions between the collector and the requester on blockchain, the proposed protocols can prevent the collector from publishing fabricated data and prevent the requester from repudiating the reward. Moreover, by engaging the nodes of the blockchain network in the process of choosing Laplacian noise, the proposed protocols can guarantee that the noisy result published by the collector satisfy both the requester's requirement for data utility and the data contributors' requirement for privacy protection. Nonetheless, as indicated by the simulation results, the interactive verifiable shuffle protocol utilized in this article makes the transaction verification process time consuming. In future study, we plan to use an noninteractive protocol so as to improve the efficiency.

## References


[1] H. Boyes, B. Hallaq, J. Cunningham, and T. Watson, "The industrial Internet of Things (IIoT): An analysis framework," *Comput. Ind.*, vol. 101, pp. 1–12, 2018.

[2] W. Khan, M. Rehman, H. Zangoti, M. Afzal, N. Armi, and K. Salah, "Industrial Internet of Things: Recent advances, enabling technologies and open challenges," *Comput. Elect. Eng.*, vol. 81, 2020, Art. no. 106522.

[3] M. R. Asghar, G. Dàn, D. Miorandi, and I. Chlamtac, "Smart meter data privacy: A survey," *IEEE Commun. Surv. Tut.*, vol. 19, no. 4, pp. 2820–2835, Oct.–Dec. 2017.

[4] H. Dai, Z. Zheng, and Y. Zhang, "Blockchain for Internet of Things: A survey," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8076–8094, Oct. 2019.

[5] W. Gao, W. G. Hatcher, and W. Yu, "A survey of blockchain: Techniques, applications, and challenges," in *Proc. 27th Int. Conf. Comput. Commun. Netw.*, July 2018, pp. 1–11.

[6] E. Jesus, V. Chicarino, C. Albuquerque, and A. Rocha, "A survey of how to use blockchain to secure Internet of Things and the stalker attack," *Secur. Commun. Netw.*, vol. 2018, pp. 1–27, Apr. 2018.

[7] D. Froelicher *et al.*, "Unlynx: A decentralized system for privacy-conscious data sharing," *Proc. Privacy Enhancing Technol.*, vol. 2017, no. 4, pp. 152–170, 2017.

[8] C. H. Liu, Q. Lin, and S. Wen, "Blockchain-enabled data collection and sharing for industrial IoT with deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3516–3526, Jun. 2019.

[9] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4177–4186, Jun. 2020.

[10] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3/4, pp. 211–407, Aug. 2014.

[11] B. C. M. Fung, K. Wang, R. Chen, and P. S. Yu, "Privacy-preserving data publishing: A survey of recent developments," *ACM Comput. Surv.*, vol. 42, no. 4, pp. 14-1–14-53, Jun. 2010.

[12] N. Mohammed, R. Chen, B. C. Fung, and P. S. Yu, "Differentially private data release for data mining," in *Proc. 17th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2011, pp. 493–501.

[13] T. Zhu, G. Li, W. Zhou, and P. S. Yu, *Differential Privacy and Applications*. Berlin, Germany: Springer, 2017.

[14] T. Elgamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE Trans. Inf. Theory*, vol. 31, no. 4, pp. 469–472, Jul. 1985.

[15] D. Hankerson, A. J. Menezes, and S. Vanstone, *Guide to Elliptic Curve Cryptography*. Berlin, Germany: Springer Science & Business Media, 2006.

[16] L. Nguyen, R. Safavi-Naini, and K. Kurosawa, "A provably secure and efficient verifiable shuffle based on a variant of the paillier cryptosystem," *J. UCS*, vol. 11, no. 6, pp. 986–1010, 2005.


Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.

[17] C. A. Neff, "Verifiable mixing (shuffling) of elgamal Pairs," VoteHere, Inc., VHTi Tech. Document, 2003. [Online]. Available: https://people.csail.mit.edu/rivest/voting/papers/Neff-2004-04-21-ElGamalShuffles.pdf

[18] E. Androulaki *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, 2018. [Online]. Available: https://doi.org/10.1145/3190508.3190538

[19] M. Castro, *et al.*, "Practical byzantine fault tolerance," in *Proc. 3rd Symp. Operating Syst. Des. Implementation*, 1999, pp. 173–186.

[20] F. D. McSherry, "Privacy integrated queries: An extensible platform for privacy-preserving data analysis," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, New York, NY, USA, 2009, pp. 19–30.

[21] A. Haeberlen, B. C. Pierce, and A. Narayan, "Differential privacy under fire," in *Proc. 20th USENIX Conf. Secur.*, 2011, pp. 1–15.

[22] A. Narayan, A. Feldman, A. Papadimitriou, and A. Haeberlen, "Verifiable differential privacy," in *Proc. 10th Eur. Conf. Comput. Syst.*, 2015. [Online]. Available: https://doi.org/10.1145/2741948.2741978

[23] Y. Zhao *et al.*, "A blockchain-based approach for saving and tracking differential-privacy cost," *IEEE Internet Things J.*, to be published, doi: 10.1109/JIOT.2021.3058209.

**Lei Xu** received the Ph.D. degree in electronic engineering from Tsinghua University (THU), Beijing, China, in 2015.

She was a Postdoctor with the Department of Computer Science and Technology, THU from July 2015 to July 2017. She is currently an Assistant Professor with the School of Computer Science and Technology, Beijing Institute of Technology. Her research interests include in privacy-preserving data publishing, privacy-preserving data sharing, and applications of blockchain technology, etc.

**Ting Bao** received the M.E. degree in computer science from the Beijing University of Technology, Beijing, China, in 2017. She is currently working toward the Ph.D. degree in local differential privacy and its application in recommendation systems with the Beijing Institute of Technology.

Her research interests include local differential privacy and its application in recommendation systems.

**Liehuang Zhu** (Member, IEEE) received the Ph.D. degree in computer science from the Beijing Institute of Technology, Beijing, China, in 2004.

He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. He has authored and coauthored more than 100 peer-reviewed journal or conference papers, including a dozen of IEEE/ACM Transactions papers (IEEE Transactions on Information Forensics and Security, IEEE Transactions on Industrial Informatics, IEEE Transactions on Vehicular Technology, IEEE Transactions on Smart Grid, *Information Sciences*, IEEE Network, Computer and Security, etc.). His research interests include security protocol analysis and design, wireless sensor networks, and cloud computing.

Dr. Zhu was the recipient of IEEE Best Paper Awards, including IWQoS'17, TrustCom'18.

Authorized licensed use limited to: Viet Nam National University Ho Chi Minh City. Downloaded on March 02,2026 at 12:28:18 UTC from IEEE Xplore. Restrictions apply.